Foundations of Q-learning

[Q-learning]

$$Q(s,a) \leftarrow Q(s,a) + \alpha\left[r + \gamma \max_{a'} Q(s',a') - Q(s,a)\right]$$

1.0 Content Block

Others Delayed Q-learning is an alternative implementation of the online Q-learning algorithm, with probably approximately correct (PAC) learning. Greedy GQ is a variant of Q-learning to use in combination with (linear) function approximation. The advantage of Greedy GQ is that convergence is guaranteed even when function approximation is used to estimate the action values. Distributional Q-learning is a variant of Q-learning which seeks to model the distribution of returns rather than the expected return of each action. It has been observed to facilitate estimate by deep neural networks and can enable alternative control methods, such as risk-sensitive control.

2.0 Content Block

Function approximation Q-learning can be combined with function approximation. This makes it possible to apply the algorithm to larger problems, even when the state space is continuous. One solution is to use an (adapted) artificial neural network as a function approximator. Another possibility is to integrate Fuzzy Rule Interpolation (FRI) and use sparse fuzzy rule-bases instead of discrete Q-tables or ANNs, which has the advantage of being a human-readable knowledge representation form. Function approximation may speed up learning in finite problems, due to the fact that the algorithm can generalize earlier experiences to previously unseen states.

3.0 Content Block

0 seconds wait time + 15 seconds fight time On the next day, by random chance (exploration), you decide to wait and let other people depart first. This initially results in a longer wait time. However, less time is spent fighting the departing passengers. Overall, this path has a higher reward than that of the previous day, since the total boarding time is now:

4.0 Content Block

Q : S × A → R {\displaystyle Q:{\mathcal {S}}\times {\mathcal {A}}\to \mathbb {R} } . Before learning begins, ■ Q {\displaystyle Q} ■ is initialized to a possibly arbitrary fixed value (chosen by the programmer). Then, at each time t {\displaystyle t} the agent selects an action A t {\displaystyle A_{t}} , observes a reward R t + 1 {\displaystyle R_{t+1}} , enters a new state S t + 1 {\displaystyle S_{t+1}} (that may depend on both the previous state S t {\displaystyle S_{t}} and the selected action), and Q {\displaystyle Q} is updated. The core of the algorithm is a Bellman equation as a simple value iteration update, using the weighted average of the current value and the new information:

5.0 Content Block

Q n e w (S t , A t) ← (1 – α ■ learning rate) · Q (S t , A t) ■ current value + α ■ learning rate · (R t + 1 ■ reward + γ ■ discount factor · max a Q (S t + 1 , a) ■ estimate of optimal future value ■ new value (temporal difference target)) {\displaystyle Q^{new}(S_{t},A_{t})\leftarrow (1-\underbrace {\alpha } _{\text{learning rate}})\cdot \underbrace {Q(S_{t},A_{t})} _{\text{current value}}+\underbrace {\alpha } _{\text{learning rate}}\cdot {\bigg (}\underbrace {\underbrace {R_{t+1}} _{\text{reward}}+\underbrace

{\gamma } _{\text{discount factor}}\cdot \underbrace {\max _{a}Q(S_{t+1},a)} _{\text{estimate of optimal future value}}} _{\text{new value (temporal difference target)}}{\bigg)}}

6.0 Content Block

History Q-learning was introduced by Chris Watkins in 1989. A convergence proof was presented by Watkins and Peter Dayan in 1992, building on Watkins' doctoral dissertation, Learning from Delayed Rewards. Eight years earlier in 1981, the same problem, under the name of "Delayed reinforcement learning," was solved by Bozinovski's Crossbar Adaptive Array (CAA). The memory matrix W = ■ w (a , s) ■ {\displaystyle W=\|w(a,s)\|} was the same as the eight years later Q-table of Q-learning. The architecture introduced the term "state evaluation" in reinforcement learning. The crossbar learning algorithm, written in mathematical pseudocode in the paper, in each iteration performs the following computation:

7.0 Content Block

5 second wait time + 0 second fight time Through exploration, despite the initial (patient) action resulting in a larger cost (or negative reward) than in the forceful strategy, the overall cost is lower, thus revealing a more rewarding strategy.

8.0 Content Block

Learning rate The learning rate or step size determines to what extent newly acquired information overrides old information. A factor of 0 makes the agent learn nothing (exclusively exploiting prior knowledge), while a factor of 1 makes the agent consider only the most recent information (ignoring prior knowledge to explore possibilities). In fully deterministic environments, a learning rate of α t = 1 {\displaystyle \alpha _{t}=1} is optimal. When the problem is stochastic, the algorithm converges under some technical conditions on the learning rate that require it to decrease to zero. In practice, often a constant learning rate is used, such as α t = 0.1 {\displaystyle \alpha _{t}=0.1} for all t {\displaystyle t} .

9.0 Content Block

Double Q-learning Because the future maximum approximated action value in Q-learning is evaluated using the same Q function as in current action selection policy, in noisy environments Q-learning can sometimes overestimate the action values, slowing the learning. A variant called Double Q-learning was proposed to correct this. Double Q-learning is an off-policy reinforcement learning algorithm, where a different policy is used for value evaluation than what is used to select the next action. In practice, two separate value functions Q A {\displaystyle Q^{A}} and Q B {\displaystyle Q^{B}} are trained in a mutually symmetric fashion using separate experiences. The double Q-learning update step is then as follows:

10.0 Content Block

where R t + 1 {\displaystyle R_{t+1}} is the reward received when moving from the state S t {\displaystyle S_{t}} to the state S t + 1 {\displaystyle S_{t+1}} , and α {\displaystyle \alpha } is the learning rate (0 < α ≤ 1) {\displaystyle (0<\alpha \leq 1)} . Note that Q n e w (S t , A t) {\displaystyle Q^{new}(S_{t},A_{t})} is the sum of three terms:

11.0 Content Block

In state s perform action a; Receive consequence state s'; Compute state evaluation ■ v (s ′) {\displaystyle v(s')} ■; Update crossbar value w ′ (a , s) = w (a , s) + v (s ′) {\displaystyle w'(a,s)=w(a,s)+v(s')} . The term "secondary reinforcement" is borrowed from animal learning theory, to model state values via backpropagation: the state value ■ v (s ′) {\displaystyle v(s')} ■ of the consequence situation is backpropagated to the previously encountered situations. CAA computes state values vertically and actions horizontally (the "crossbar"). Demonstration graphs showing delayed reinforcement learning contained states (desirable, undesirable, and neutral states), which were computed

by the state evaluation function. This learning system was a forerunner of the Q-learning algorithm. In 2014, Google DeepMind patented an application of Q-learning to deep learning, entitled "deep reinforcement learning" or "deep Q-learning," that can play Atari 2600 games at expert human levels.